NgRx SignalStore Complete Cheatsheet

withState · withComputed · withMethods · withHooks · withEntities | Example: Like Post + Order Feed

Coupa L1 Prep

Sumit · Senior Angular

NgRx Signals 17+ | Angular Signals | No Actions/Reducers | Senior Level

1 withState — 2 withComputed — 3 withMethods — 4 withHooks — 5 withEntities — 6 COMPONENT

withState · define initial state shape

```
// store/post.store.ts
import { signalStore, withState,
  withComputed, withMethods, withHooks }
  from '@ngrx/signals';

// 1. Define your state interface
interface PostState {
  posts:   Post[];
  loading: boolean;
  error:   string | null;
  filter:  'all' | 'liked';
}

// 2. Initial state values
const initialState: PostState = {
  posts:   [],
  loading: false,
  error:   null,
  filter:  'all',
};

// 3. Create the store (Injectable service)
export const PostStore = signalStore(
  withState(initialState),
  // ... more features below
);
```

withComputed · derived signals, auto-memoized

```
withComputed((store) => ({

  // Derived from state — like NgRx selectors
  likedPosts: computed(() =>
    store.posts().filter(p => p.liked)),

  totalCount: computed(() =>
    store.posts().length),

  filteredPosts: computed(() => {
    const f = store.filter();
    return f === 'liked'
      ? store.posts().filter(p => p.liked)
      : store.posts();
  }),

  // Combine multiple signals
  isEmpty: computed(() =>
    !store.loading() && store.posts().
      length === 0),
})),
```

withHooks · lifecycle — init & destroy

```
withHooks({
  // Runs when store is first injected
  onInit(store) {
    // Load data on store init
    store.loadPosts();

    // React to signal changes (like effect())
    effect(() => {
      const filter = store.filter();
      console.log('Filter changed:', filter);
      store.loadPosts();  // reload on filter change
    });
  },

  // Runs on store destroy (component unmount)
  onDestroy(store) {
    store.cleanup();
  },
}),
```

withMethods · actions + API calls in one place

```
withMethods((store) => ({

  // Load posts from API
  loadPosts: async () => {
    patchState(store, { loading: true, error: null });
    try {
      const posts = await inject(PostApi).getAll();
      patchState(store, { posts, loading: false });
    } catch(err: any) {
      patchState(store, { loading: false,
        error: err.message });
    }
  },

  // Optimistic Like Post (same as NgRx example)
  likePost: async (postId: string) => {
    const post = store.posts().
      find(p => p.id === postId);
    if (!post) return;
    const prevLikes = post.likes;

    // 1. Optimistic update immediately
    patchState(store, { posts:
      store.posts().map(p =>
        p.id===postId ? {...p, likes: p.likes+1} : p)
    });

    try {
      await inject(PostApi).like(postId);
    } catch {
      // 2. Rollback on failure
      patchState(store, { posts:
        store.posts().map(p =>
          p.id===postId ? {...p, likes: prevLikes} : p)
      });
    }
  },

  // Simple state update method
  setFilter(filter: 'all' | 'liked') {
    patchState(store, { filter });
  },

  // Cleanup (called from onDestroy)
  cleanup() {
    // cancel WS, clear timers etc.
  },
})),
```

withEntities · built-in CRUD for collections

```
import { withEntities, setAllEntities,
  updateEntity, removeEntity }
  from '@ngrx/signals/entities';

export const OrderStore = signalStore(
  withEntities<Order>(),  // auto: ids[], entities{}
  withMethods((store) => ({
    loadOrders: async () => {
      const orders = await inject(OrderApi).getAll();
      patchState(store, setAllEntities(orders));
    },
    approveOrder(id: string) {
      patchState(store,
        updateEntity({ id, changes: { status: 'approved' }}));
    },
    deleteOrder(id: string) {
      patchState(store, removeEntity(id));
    },
  }))
);

// Auto-generated signals: store.entities(), store.ids()
// store.entityMap()  — O(1) lookup by id
```

COMPONENT · inject store, use signals directly

```
    @for (post of store.filteredPosts(); track post.id) {
      <app-post [post]="post"
```

withState
Define shape once. patchState() to update.

withComputed
Like selectors — auto memoized via computed().

withMethods
All logic here. inject() works inside methods.

withHooks
onInit = constructor. onDestroy = cleanup.

withEntities
Built-in CRUD. Never manage ids[] manually.